Spark Word Count
Python & Scala

A Step-by-Step MapReduce Tutorial

PySpark & Spark Shell Examples

Tutorial: Spark Word Count (Python & Scala)

The Word Count example is the "Hello World" of Apache Spark. It demonstrates how to read a text file, transform the data using MapReduce concepts, and output the results.

1. PySpark Word Count (Python)

Create a file named word_count.py:

```
  from pyspark.sql import SparkSession

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("WordCount") \
    .getOrCreate()

# Read text file
# Replace 'data.txt' with your file path
text_file = spark.read.text("data.txt").rdd.map(lambda r: r[0])

# Perform Word Count
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)

# Collect and print results
output = counts.collect()
for (word, count) in output:
    print(f"{word}: {count}")

# Stop Spark Session
spark.stop()
```

How to Run:

```
  spark-submit --master spark://your-master-ip:7077 word_count.py
```

2. Spark Shell Word Count (Interactive)

You can run this directly in the spark-shell (Scala):

```
  // Read file
val textFile = sc.textFile("data.txt")

// Transform and count
val counts = textFile.flatMap(line => line.split(" "))
                  .map(word => (word, 1))
                  .reduceByKey(_ + _)

// Save output
counts.saveAsTextFile("output_directory")
```

3. Detailed Code Explanation

* **`SparkSession`**: The entry point to Spark functionality.
* **`read.text()`**: Loads the file as a Dataset/DataFrame.
* **`flatMap()`**: Splits each line into individual words. A single input line produces multiple output words.
* **`map()`**: Transforms each word into a key-value pair `(word, 1)`.
* **`reduceByKey()`**: Groups all pairs with the same key (word) and sums their values.
* **`collect()`**: Pulls the results from the cluster nodes back to the driver node (use with caution on large datasets).

4. Tips for Large Scale

* **Use `saveAsTextFile`**: Instead of `collect()`, save results to HDFS, S3, or a local directory to handle massive data.
* **Data Cleaning**: Use `.filter(lambda x: x != "")` to remove empty strings after splitting.
* **Lowercasing**: Use `.map(lambda x: x.lower())` for case-insensitive counting.